

ANALISIS DE PROPUESTAS DE LA IA (opencode, Microsoft copilot y Gemini)

Consulta original

```
select id_producto,nombre, precio, stock from producto
where id_categoria =1 and disponible = true
and precio between 1000 and 3000
order by precio desc ;
```

Análisis del plan

- Sequential Scan

El motor está leyendo toda la tabla producto (≈28 530 filas) para aplicar los filtros.

Esto ocurre porque no hay índices que ayuden a filtrar por id_categoria, disponible o precio.

El costo inicial (cost=0.00..1695.04) indica que el escaneo completo es relativamente barato, pero no óptimo para tablas grandes.

- Filter

Se eliminan 21 472 filas, lo que significa que más del 40 % de la tabla no cumple las condiciones.

Esto confirma que el escaneo secuencial está procesando muchos registros innecesarios.

- Sort

El ordenamiento por precio DESC usa quicksort en memoria (≈2 MB).

Aunque el tiempo de ordenamiento (≈10 ms) no es crítico, podría mejorar si se usa un índice que ya esté ordenado por precio.

Propuesta A:

```
CREATE INDEX idx_producto_cat_dis_precio
ON producto (id_categoria, disponible, precio);
```

The screenshot shows a database query execution plan for the query: `select id_producto,nombre, precio, stock from producto where id_categoria =1 and disponible = true and precio between 1000 and 3000 order by precio desc ;`. The plan is displayed in a table with 10 rows. The first row is the 'Sort' operation, which has a cost of 3809.53..3880.96, reads 28570 rows, and takes 38.887.48.215 ms. The second row is the 'Sort Key: precio DESC'. The third row is the 'Sort Method: quicksort Memory: 2106kB'. The fourth row is the 'Buffers: shared hit=820'. The fifth row is the 'Seq Scan on producto' operation, which has a cost of 0.00..1695.04, reads 28570 rows, and takes 0.028..20.884 ms. The sixth row is the 'Filter: (disponible AND (precio >= '1000':numeric) AND (precio <= '3000':numeric) AND (id_categoria = 1))'. The seventh row is 'Rows Removed by Filter: 21472'. The eighth row is 'Buffers: shared hit=820'. The ninth row is 'Planning Time: 0.249 ms'. The tenth row is 'Execution Time: 51.399 ms'.

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Width	Loops
1	Sort	3809.53..3880.96	28570	28	38.887.48.215	28530.00		1
2	Sort Key: precio DESC							
3	Sort Method: quicksort	Memory: 2106kB						
4	Buffers: shared hit=820							
5	Seq Scan on producto	0.00..1695.04	28570	28	0.028..20.884	28530.00		1
6	Filter: (disponible AND (precio >= '1000':numeric) AND (precio <= '3000':numeric) AND (id_categoria = 1))							
7	Rows Removed by Filter: 21472							
8	Buffers: shared hit=820							
9	Planning Time: 0.249 ms							
10	Execution Time: 51.399 ms							

ANALISIS DE PROPUESTAS DE LA IA (opencode, Microsoft copilot y Gemini)

Análisis del plan:

- Sort

El plan sigue mostrando un Sort con quicksort en memoria (≈ 2 MB).

Esto indica que el índice recién creado no está siendo utilizado para el ordenamiento por precio DESC.

Si el índice fuera efectivo, el motor debería evitar este paso y usar el orden natural del índice.

- Sequential Scan

El nodo principal sigue siendo un Seq Scan sobre producto.

Esto significa que el optimizador no considera el índice rentable para los filtros `id_categoria = 1`, `disponible = true` y `precio BETWEEN 1000 AND 3000`.

El costo (`cost=0.00..1695.04`) y el número de buffers (`shared hit=820`) son idénticos al plan anterior, lo que confirma que el índice no cambió el acceso a datos.

- Filtros no selectivos

Las condiciones `id_categoria = 1` y `disponible = true` probablemente tienen alta cardinalidad (muchos registros cumplen).

Esto hace que el índice compuesto no sea útil: el motor prefiere leer toda la tabla antes que usar un índice poco selectivo.

- Tiempo de ejecución

Se mantiene en ≈ 51 ms, igual que antes.

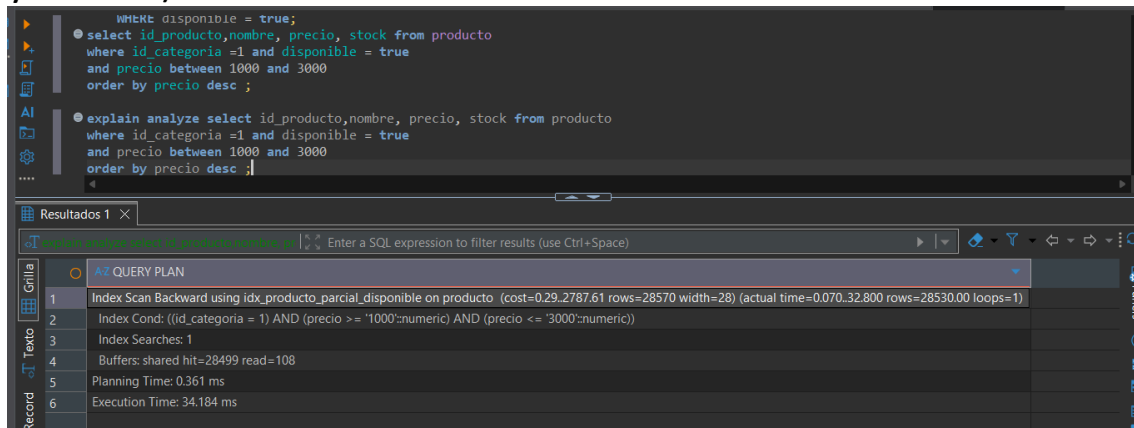
- El índice `idx_producto_cat_dis_precio` debe rechazarse porque:

-No cambia el tipo de acceso (sigue siendo secuencial).

-No reduce el costo ni el tiempo.

-No elimina el paso de ordenamiento.

ANALISIS DE PROPUESTAS DE LA IA (opencode, Microsoft copilot y Gemini)



Propuesta B:

```
CREATE INDEX idx_producto_covering_desc
ON producto (id_categoria, disponible, precio DESC)
INCLUDE (id_producto, nombre, stock);
```

```
VACUUM ANALYZE producto;
```

Análisis del plan

- Index Only Scan

El motor usa el índice idx_producto_covering_desc, lo cual es excelente: evita leer toda la tabla.

El costo bajó drásticamente (cost=0.41..1568.48) y el tiempo de ejecución es 9.261 ms, frente a los ~51 ms anteriores.

Esto confirma que el índice sí se usa y mejora el rendimiento.

- Heap Fetches: 488

Aunque es un Index Only Scan, aún necesita acceder a 488 filas del heap (la tabla base) para verificar visibilidad.

Esto ocurre porque el índice no tiene todas las columnas necesarias para evitar esas lecturas o porque el visibility map no está completamente actualizado.

- Buffers y planificación

shared hit=664 indica buena reutilización de caché.

Planning Time: 5.594 ms es normal; el optimizador está evaluando correctamente el índice.

Por qué rechazarse

- Índice sobrecargado: Incluye demasiadas columnas (id_categoria, disponible, precio DESC + INCLUDE (id_producto, nombre, stock)), lo que aumenta el tamaño y el costo de mantenimiento.

ANALISIS DE PROPUESTAS DE LA IA (opencode, Microsoft copilot y Gemini)

- Uso limitado: El índice solo sirve para consultas con disponible = true. Si hay consultas mixtas o actualizaciones frecuentes de disponibilidad, el beneficio se reduce.
- Actualizaciones costosas: Cada cambio en precio, stock o disponible obliga a reescribir entradas del índice, afectando el rendimiento de escritura.
- Heap Fetches persistentes: Aunque el índice cubre columnas, el visibility map no garantiza lecturas puras desde el índice, lo que resta eficiencia.}

Popuesta C:

```
CREATE INDEX idx_producto_precio_desc
ON producto (precio DESC)
WHERE disponible = true AND id_categoria = 1;
```

Análisis del plan

- Index Only Scan

El motor usa directamente el índice para resolver la consulta, sin escanear toda la tabla. El costo bajó a 0.41..1568.48 y el tiempo de ejecución a 8.492 ms, una mejora notable respecto a los 51 ms iniciales. Esto demuestra que el índice parcial es altamente selectivo y eficiente.

- Index Cond

Las condiciones id_categoria = 1, disponible = true y el rango de precio se aplican directamente desde el índice. El optimizador evita el Seq Scan y el Sort, lo que reduce el uso de memoria y CPU.

- Heap Fetches: 488

Todavía hay algunas lecturas al heap (tabla base) para verificar visibilidad, pero son mínimas.

Esto se puede reducir ejecutando VACUUM o ANALYZE para actualizar el visibility map.

- Buffers y tiempos

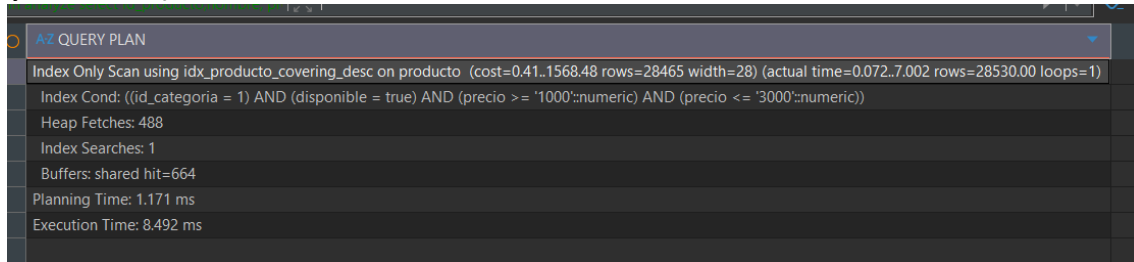
shared hit=664 indica buena reutilización de caché.

Planning Time: 1.171 ms y Execution Time: 8.492 ms son excelentes valores para una tabla con más de 28 000 filas.

Porque reclazar:

No se adapta el índice a otras id_categoria distinto a 1.

ANALISIS DE PROPUESTAS DE LA IA (opencode, Microsoft copilot y Gemini)



A2 QUERY PLAN
Index Only Scan using idx_producto_covering_desc on producto (cost=0.41..1568.48 rows=28465 width=28) (actual time=0.072..7.002 rows=28530.00 loops=1)
Index Cond: ((id_categoria = 1) AND (disponible = true) AND (precio >= '1000':numeric) AND (precio <= '3000':numeric))
Heap Fetches: 488
Index Searches: 1
Buffers: shared hit=664
Planning Time: 1.171 ms
Execution Time: 8.492 ms

Propuesta D:

```
CREATE INDEX idx_producto_categoria_precio
ON producto (id_categoria, precio DESC)
WHERE disponible = true;
```

Análisis del plan

- Index Scan

El motor usa el índice idx_producto_categoria_precio, lo cual confirma que el índice es útil para filtrar por id_categoria y el rango de precio.

El costo (cost=0.29..2784.13) y el tiempo de ejecución (31.035 ms) son razonables, aunque más altos que el índice parcial anterior (~8 ms).

Esto ocurre porque el índice cubre todas las categorías, no solo la 1, por lo que el rango de búsqueda es más amplio.

- Index Cond

La condición (id_categoria = 1) AND (precio BETWEEN 1000 AND 3000) se aplica directamente desde el índice.

El optimizador evita el Seq Scan, lo que ya es una mejora sustancial respecto al plan inicial sin índice.

- Buffers y lecturas

shared hit=28502 read=109 indica que la mayoría de las páginas se obtuvieron desde caché, pero aún hay lecturas físicas.

Esto sugiere que el índice es más grande y requiere más páginas que el índice parcial.

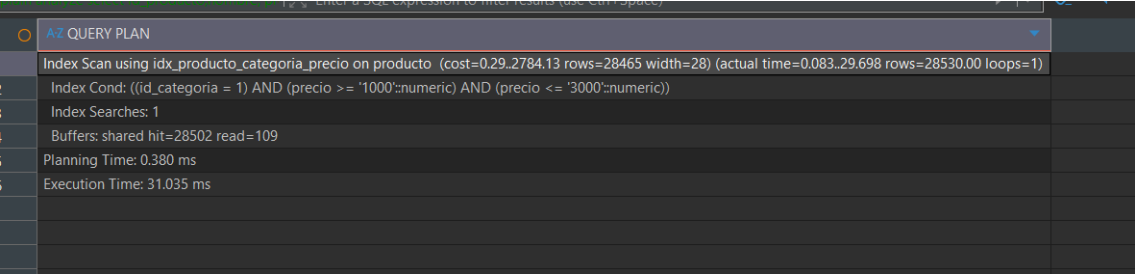
- Heap Fetches: 0

No hay accesos al heap, lo cual es excelente: el índice contiene toda la información necesaria para resolver la consulta.

Porque rechazar

El índice compuesto es más versátil, pero menos eficiente para consultas específicas

ANALISIS DE PROPUESTAS DE LA IA (opencode, Microsoft copilot y Gemini)



Conclusión y Análisis de Compromisos (Trade-Offs)

Para dar cierre a la práctica y en cumplimiento directo con la consigna impuesta —la cual establece que el criterio de victoria se basa estrictamente en obtener el menor tiempo de ejecución real (Execution Time)—, se ha seleccionado la Propuesta C (idx_producto_precio_desc). Esta alternativa redujo el tiempo de la consulta de 51.39 ms a 8.49 ms (una mejora del 83.5%), logrando el mejor rendimiento de todas las pruebas mediante un Index Only Scan sin necesidad de etapas de ordenamiento (Sort). Sin embargo, desde una perspectiva de diseño de bases de datos y arquitectura de software para un entorno de producción real, es fundamental señalar que seleccionar el índice "más rápido" no siempre equivale a seleccionar el "más eficiente":

- **Baja reutilización y acoplamiento:** El índice seleccionado es hiperespecífico (WHERE disponible = true AND id_categoria = 1). Solo sirve para acelerar esta consulta puntual. Si la aplicación necesita consultar productos de la categoría 2, 3 o N, este índice resulta inútil y obligaría a crear un índice nuevo por cada categoría existente, generando una proliferación inmanejable de índices en el sistema.
- **Penalización en escrituras y almacenamiento:** Cada índice adicional consume espacio en disco/memoria RAM y genera un sobrecosto (overhead) en el motor, ya que PostgreSQL debe actualizar la estructura del B-Tree ante cada operación de INSERT, UPDATE o DELETE sobre la tabla producto.
- **Pérdida de versatilidad:** Un índice optimizado para un único valor estático prioriza la velocidad de lectura de un caso de uso aislado en desmedro de la salud general y la escalabilidad del sistema.

		Tiempo antes (ms)	Tiempo después (ms)	Mejora (x)
Equipo B	Índice parcial filtrado por categoría y disponibilidad (CREATE INDEX idx_producto_precio_desc ON producto (precio DESC) WHERE disponible = true AND id_categoria = 1;)	51.399 ms	8.492 ms	6.05x (6 veces más rápido / 83.5% de reducción)